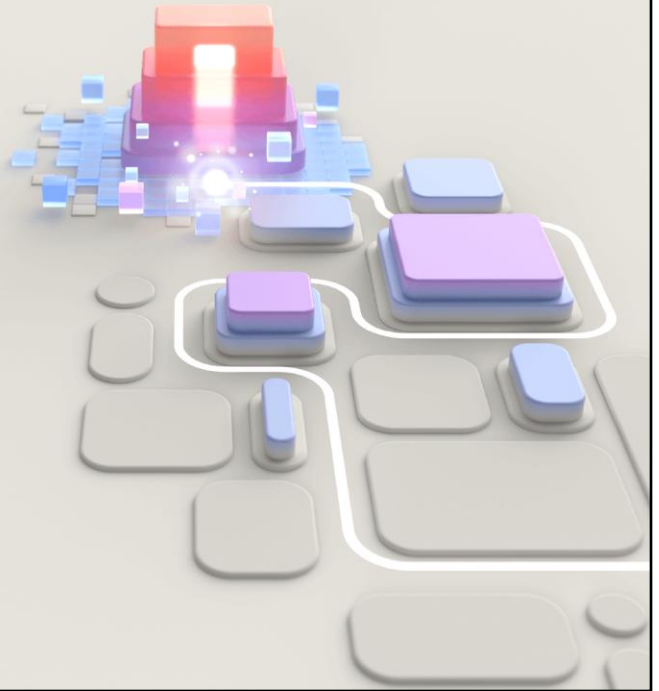




PL-400T00

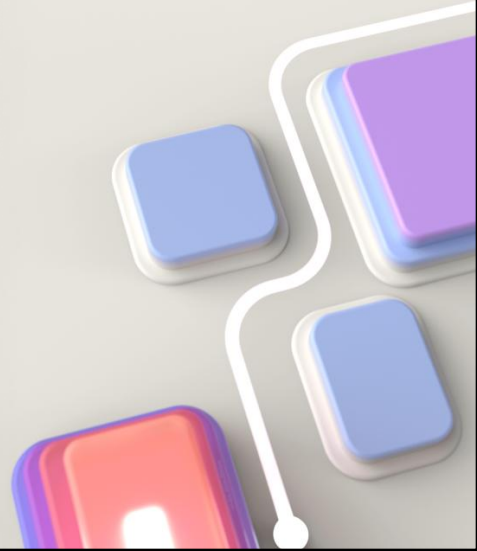
Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.

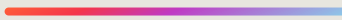


Create code components with the Power Apps Component Framework

© Copyright Microsoft Corporation. All rights reserved.



Modules



- Get started with Power Apps component framework

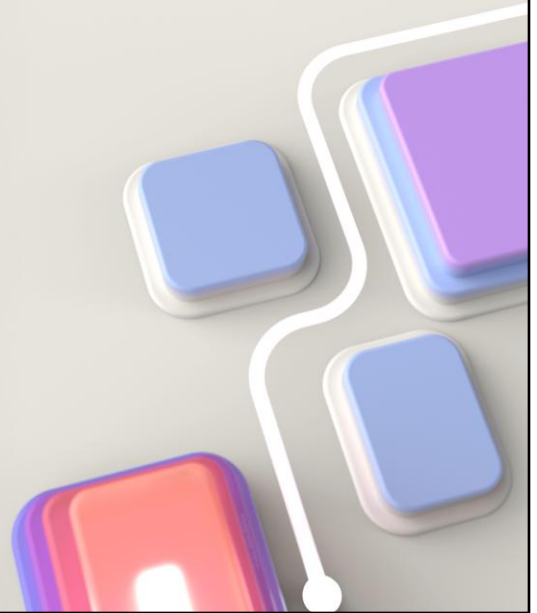
© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Get started with Power Apps component framework

Module 1: Get started with Power Apps component framework

© Copyright Microsoft Corporation. All rights reserved.



In this module, you will:

- Learn about Power Apps component framework architecture
- Learn about Power Apps component tooling

Learning Objectives

After completing this module, you will be able to:

- 1 Power Apps component framework architecture
- 2 Power Apps component tooling

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Get started with Power Apps component framework

<https://learn.microsoft.com/training/modules/get-started-component-framework>



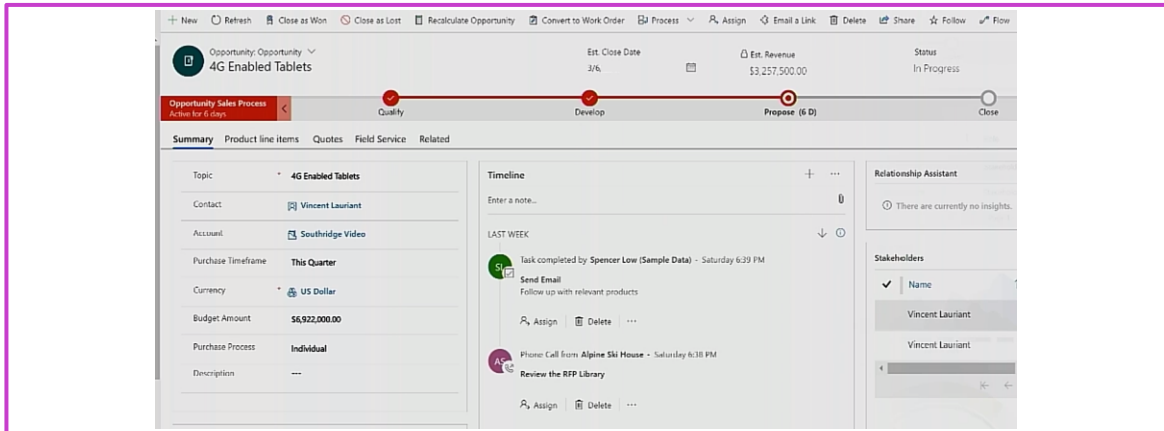
© Copyright Microsoft Corporation. All rights reserved.

Get started with Power Apps component framework

<https://learn.microsoft.com/training/modules/get-started-component-framework>

Introduction to Power Apps component framework

Microsoft Power Apps component framework helps you create reusable components that can be used within your Power Apps applications.



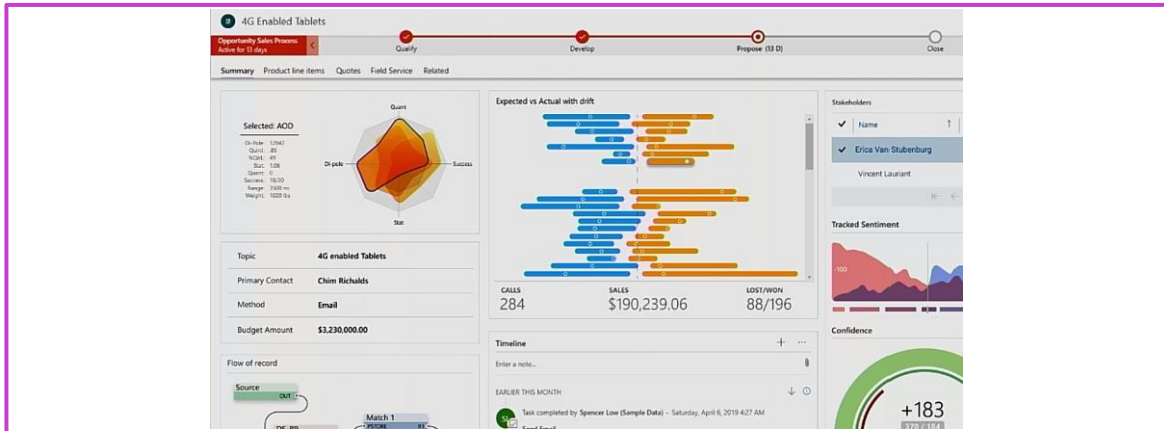
© Copyright Microsoft Corporation. All rights reserved.

Microsoft Power Apps component framework helps you create reusable components that can be used within your Power Apps applications. The component framework empowers developers and app makers to build code components when the out-of-the-box components don't fit an app maker's needs. Before the Power Apps component framework existed, people had to rely on HTML web resources to provide any form of custom presentation to a Power Apps' form. Now, you can use a more modernized framework that allows an abundance of capabilities to be exposed to your app that would otherwise be impossible to access or, even worse, be unsupported by Microsoft.

For example, the existing screen might render similar to the following image.

Introduction to Power Apps component framework

However, if you reconfigured your app to use custom Power Apps components, your app might look something like the following image.



© Copyright Microsoft Corporation. All rights reserved.

Microsoft has significant investment in ensuring that Power Apps components are built on top of a robust framework that supports modern web practices.

What are PCF Code components

- 1 Reusable rich user interface controls
- 2 Created by developers
- 3 Developed using HTML, CSS, and TypeScript
- 4 Packaged as solutions
- 5 Can be used in model-driven apps, canvas apps, and Power Pages sites

© Copyright Microsoft Corporation. All rights reserved.

Microsoft using them for their own development e.g. editable grids, new schedule board, new FS mobile app

Power Apps code component framework advantages

A few of the advantages that you are afforded as a results are:

- Access to a rich set of framework APIs that expose capabilities like component lifecycle management, contextual data, and metadata
- Support of client frameworks such as React and AngularJS
- Seamless server access through Web API, utility and data formatting methods, device features like camera, location, and microphone, in addition to easy-to-invoke UX elements like dialogs, lookups, and full-page rendering
- Optimization for performance
- Reusability
- Use of responsive web design principles to provide an optimal viewing and interaction experience for any screen size, device, or orientation
- Ability to bundle all files into a single solution file

© Copyright Microsoft Corporation. All rights reserved.

Microsoft has significant investment in ensuring that Power Apps components are built on top of a robust framework that supports modern web practices. A few of the advantages that you are afforded as a result are:

- Access to a rich set of framework APIs that expose capabilities like component lifecycle management, contextual data, and metadata
- Support of client frameworks such as React and AngularJS
- Seamless server access through Web API, utility and data formatting methods, device features like camera, location, and microphone, in addition to easy-to-invoke UX elements like dialogs, lookups, and full-page rendering
- Optimization for performance
- Reusability
- Use of responsive web design principles to provide an optimal viewing and interaction experience for any screen size, device, or orientation
- Ability to bundle all files into a single solution file

Types of components that you can add

- 1 Field: A custom control for a column on a form

- 2 Dataset: A custom grid (or subgrid) to display data in a tabular format

- 3 A component that displays content from external services

© Copyright Microsoft Corporation. All rights reserved.

It might be said that if you can build it in HTML+JavaScript, you can build it as a Power Apps component. However, common types of components that you might want to use in a Power Apps application might be:

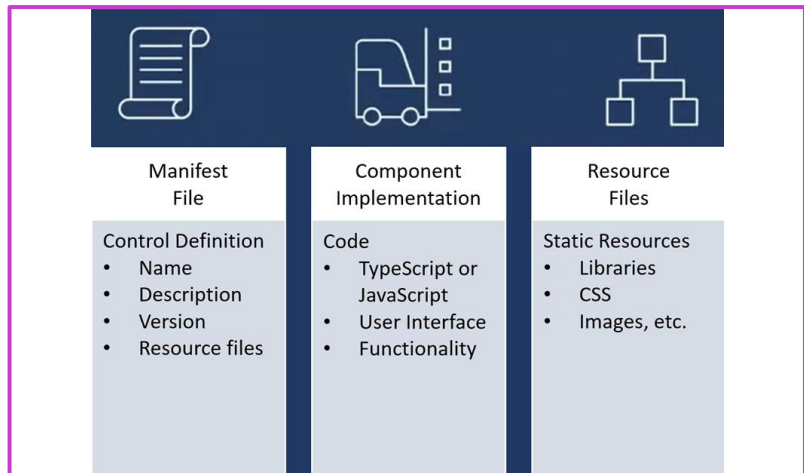
- A custom control for a column on a form
- A custom grid (or subgrid) to display data in a tabular format
- A component that displays content from external services

Types of components that you can add

- **Field** - A custom control for a field on a form. For example, instead of a simple text box to input a number, a custom code component could be used to render a slider. The slider could have custom business logic that limited the stops on the slider based on other data available and bound to the component.
- **Dataset** - A custom control to display rows of data. For example, instead of a common grid with rows and columns to display today's appointments a daily scheduler code component could be built. The code component could include features like drag and drop to reschedule appointments.

Component composition

Code components are implemented using HTML, CSS, and TypeScript



© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/component-framework/custom-controls-overview>

A PCF component is represented as three key areas:

- Manifest file
- Component implementation
- Resource files that might be needed by the component

Power Apps component life cycle

When developing a component, you are expected to implement the life cycle methods that are shown in the following table.

Method	Description
init	This method is used to initialize the component instance. Components can kick off remote server calls and other initialization actions in this method. Dataset values cannot be initialized with this method; you will need to use the updateView method for this purpose.
updateView	This method will be called when any value in the component's property bag has changed.
getOutputs	Called by the framework prior to the receipt of new data. Use this method when dynamically managing bound properties in a control. (Optional)
destroy	Invoked when the component is to be removed from the DOM tree. Used for cleanup and to release any memory that the component is using.

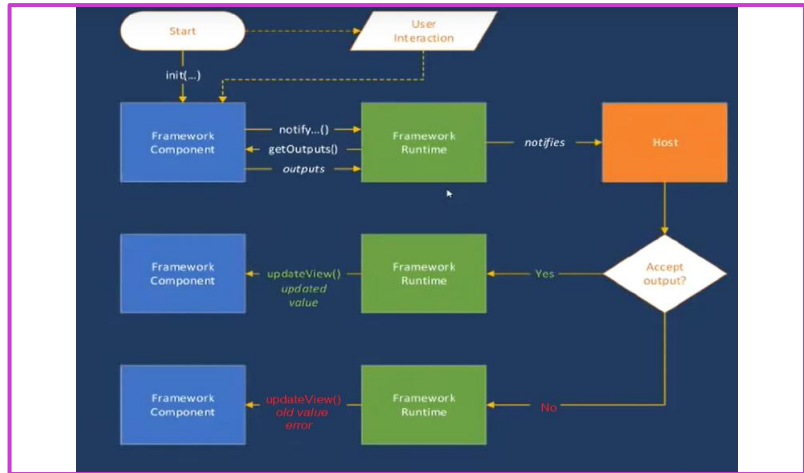
© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/component-framework/reference/control-Methods>

- Init
 - Required.
 - This method is used to initialize the component instance. Components can kick off remote server calls and other initialization actions in this method.
- updateView
 - Required.
 - This method will be called when any value in the component's property bag has changed.
- getOutputs
 - Optional.
 - Called by the framework prior to the receipt of new data.
 - Use this method when dynamically managing bound properties in a control.
- destroy
 - Required.
 - Invoked when the component is to be removed from the DOM tree.
 - Used for cleanup and to release any memory that the component is using.

Power Apps component life cycle

These methods are invoked through a Framework Runtime process in a standardized life cycle, as shown in the following illustration.



© Copyright Microsoft Corporation. All rights reserved.

At the top of the image, the framework calls the `init()` function for your component. If your component is interactive, you'll also need to handle notification to the Host that the component's output has changed by calling the `notifyOutputChanged` method. The framework runtime will then call your implemented `getOutputs` function, which represents all of your component's outputs, and then return that to the runtime. The runtime will then notify the Host of that value, which will perform validation on the output. If the output is found to be valid, it will call the `updateView` method on your component. If it isn't valid for whatever reason (that is, a business rule found the new output to be invalid), it will call your `updateView` method and pass the old value along with an error message.

<https://learn.microsoft.com/power-apps/developer/component-framework/custom-controls-overview>

Page load

When the page loads, the application requires an object to work. Using the data from the manifest file

When the page is ready, it initializes the component by calling the `init` method with a set of parameters

User changes data

When a user interacts with your components to change data, your component must call the method passed in as `notifyOutputChanged` parameter in the `init` method. When you use this method, the platform then responds by calling the `getOutputs` method. The `getOutputs` method returns values that have the changes made by the user. For a field component, this would typically be the new value for the component.

App changes data

If the platform changes the data, it calls out the `updateView` method of the component and passes the new context object as a parameter. This method should be implemented to update the values displayed in the component.

Page close

Developers should implement the `destroy` method, which is called when the page closes, to remove any cleanup code such as event handlers.

Power Apps component tooling

Power Apps CLI

Power Apps component
CLI commands

Choose an IDE

© Copyright Microsoft Corporation. All rights reserved.

Microsoft has built a robust CLI framework that exposes scaffolding methods that rapidly expedite your ability to build custom Power Apps components. It provides simple-to-use control templates along with built-in validations to catch issues prior to deployment of your control.

- Power Platform CLI
- .NET Framework 4.6.2
- .NET Build Tools
 - MSBuild
- Visual Studio Code
- Node.js
- XrmToolBox
 - PCF Builder

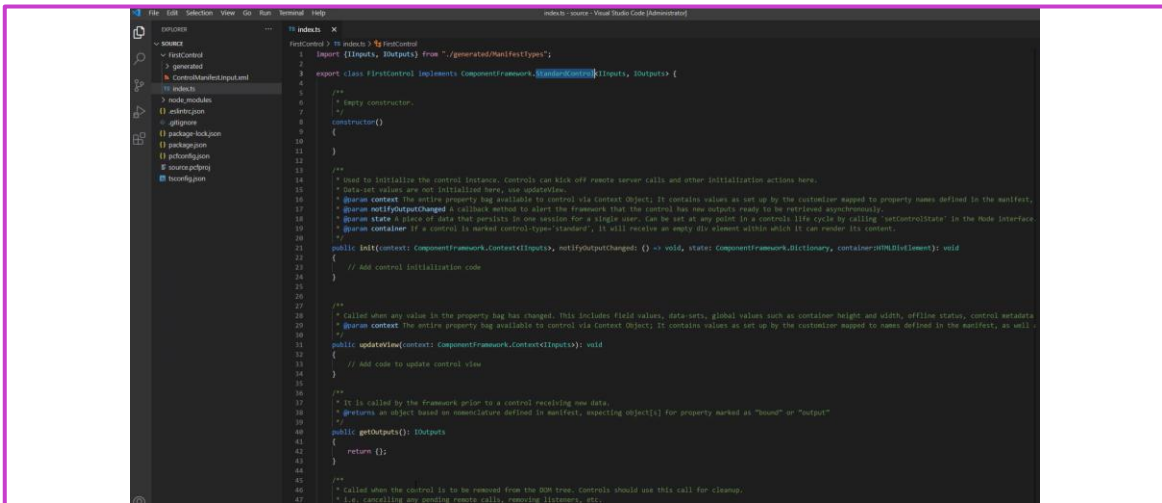
To use the Power Apps CLI, you'll first need to install [Npm](#) (comes with Node.js) or [Node.js](#) (comes with npm). We recommend that you install LTS (Long Term Support) version

For additional prerequisites and a link to the CLI's installation bits, see [Install Microsoft Power Apps CLI](#).

The steps to install CLI are covered in more detail when you learn how to build a Power Apps component.

While we recommend that you use Visual Studio or Visual Studio Code to write your components, you can use any IDE of your choice to build custom Power Apps components.

Demo: Power Apps code component



© Copyright Microsoft Corporation. All rights reserved.

<https://docs.microsoft.com/en-us/learn/modules/get-started-component-framework/4a-demo>

Or just open this video: <https://www.microsoft.com/en-us/videooplayer/embed/RWDP7c?postJsllMsg=true>

<https://www.microsoft.com/en-us/videooplayer/embed/RWDP7c>

Test and debug code components

PowerApps component framework Test Environment

484.00

TSLinearInputComponent

Context Inputs

Form Factor
Web

Component Container Width
Component Container Height

Data Inputs

Property
sliderValue

Value
484

Type
Decimal

© Copyright Microsoft Corporation. All rights reserved.

<https://docs.microsoft.com/en-us/learn/modules/build-power-app-component/3-debug-code-component>

Components and Microsoft Dataverse Security

- Remember that code components are designed to be app-agnostic and useable from the following services:
 - Model-driven apps
 - Canvas apps
 - Power Pages sites
- Consider security in the following scenarios where a PCF control is deployed:
 - Manifest with the **uses-feature WebAPI** setting enabled
 - Canvas app using the Dataverse connector
 - Power pages site on a list / form
- Configure custom security roles with the discrete privileges required; do **not** just assign **System Administrator** role
 - Power Pages sites requires [Web roles](#) and [Table permissions](#) instead
- Consider the following factors:
 - Privilege type – Create, Read, Update, Delete, Append, Append To, Assign & Share
 - Depth – Is Organization level sufficient?
 - How are the roles assigned?

© Copyright Microsoft Corporation. All rights reserved.

The main value behind using PCF controls in the first instance is the fact they are versatile and can be re-used. As such, it could be that you build a control that never involves or requires interactivity with Microsoft Dataverse. For example, you could have a control in a canvas app that connects to SharePoint that renders data in a particular way. In this scenario, considering additional security factors may not be necessary.

However, there will be situations where security is a concern, such as:

- **Manifest with the uses-feature WebAPI setting enabled:** In this context, any calls to the Dataverse Web API will execute in the context of the user operating the control. This means that if the user doesn't have permission, then they will receive a relevant permissions error. Conversely, if the user has been assigned *too many* privileges, then the control could provide an indirect mechanism to unwanted behaviour. In this scenario, you should carefully review the security role assignments.
- **Canvas app using the Dataverse connector:** If the app is shared to users using the non-legacy connector, then again, the user's individual security role assignments will apply.
- **Power Pages site:** The security model for portal sites differs and requires additional configuration via Web Roles and Entity Permissions. Consider as well whether the site has authentication enabled or not.

In these scenarios, take care to configure roles with precise privileges required per user, role, department etc.

Additional Considerations

- Have the correct privilege types been assigned based on the Xrm.WebApi request your control is performing?
- Is the depth level for each privilege scoped accordingly? Will User level suffice or is Organization level really necessary?
- How are security roles assigned? To individual users or via teams? Do you have sufficient permissions to do this yourself or do you need to escalate to an admin?

Security is often an afterthought and common trap for developers to fall into late in a project cycle. Take care and attention to consider all facets of security upfront to provide problems from occurring as your solution is being tested and deployed out.

Learning Path 5 practice labs



Power Apps Component Framework code components

Lab 7: Power Apps Component Framework

© Copyright Microsoft Corporation. All rights reserved.

In this lab you will develop a custom component control using the Power Apps Component Framework (PCF). This component will be used to visualize the permit inspection history timeline. As you build the component, you will see how to use prescriptive interfaces of the framework to interact with the hosting form data.

To speed things up we will use a community timeline library to render the visualization. When you build such controls, you can either follow the same procedure or use popular frameworks like React to completely control the visualization that the component will render.

Summary



- The Power Apps component framework is a robust framework built on modern web technologies
- Developers can write reusable code components when out-of-the-box components may not suit an app maker's needs
- Code components can be developed and tested on a computer and then deployed as solution components

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Get started with Power Apps component framework



© Copyright Microsoft Corporation. All rights reserved.